

B28: SQL SuperHero Program : Session 15

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

9 March 2025 at 06:49

```
CREATE TABLE AB_EMPLOYEE  
( EMP_ID VARCHAR2(5 ),  
  EMP_NAME VARCHAR2(20 ),  
  DEPT_ID VARCHAR2(5 ),  
  EXPERTISE VARCHAR2(50 ),  
  SALARY NUMBER(10,2),  
  RESULTS VARCHAR2(10 )  
);
```

```
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','SCIENCE','PASS',50000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','ENGLISH','PASS',50000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','MATH','PASS',50000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('107','AMARESH','2','MATH','PASS',50000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('107','AMARESH','2','ENGLISH','PASS',50000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('105','JYOTT','3','MATH','FAIL',75000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('105','JYOTT','3','ENGLISH','PASS',75000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('7003','NISHAD','2','ENGLISH','FAIL',70000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('7003','NISHAD','2','MATH','PASS',70000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('6003','RAKESH','2','MATH','PASS',50556);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('6003','RAKESH','2','ENGLISH','FAIL',50556);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('104','RAVI','2','MATH','PASS',70000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('104','RAVI','2','ENGLISH','PASS',70000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('106','REDDY','2','MATH','FAIL',80000);  
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('106','REDDY','2','ENGLISH','PASS',80000);
```

when dept id changes start again

```
7 SELECT EMP_ID, DEPT_ID,  
8 ROW_NUMBER() OVER (partition by DEPT_ID ORDER BY SALARY DESC ) AS ROW_NUM  
9 FROM AB_EMPLOYEE  
10
```

Script Output * Query Result 2 * Query Result 3 * Query Result 4 * Query Result 5 *

All Rows Fetched: 15 in 0.509 seconds

EMP_ID	DEPT_ID	ROW_NUM
1 5003	1	1
2 5003	1	2
3 5003	1	3
4 106	2	1
5 106	2	2
6 104	2	3
7 104	2	4
8 7003	2	5
9 7003	2	6
10 6003	2	7
11 6003	2	8
12 107	2	9
13 107	2	10
14 105	3	1
15 105	3	2

for 1

start again for 2

Again for 3

```
SELECT EMP_ID, EXPERTISE,  
ROW_NUMBER() OVER (partition by EXPERTISE ORDER BY SALARY ) AS ROW_NUM  
FROM AB_EMPLOYEE;
```

Expertise change, assign again

```

7 SELECT EMP_ID, EXPERTISE,
8 ROW_NUMBER() OVER (partition by EXPERTISE ORDER BY SALARY ) AS ROW_NUM
9 FROM AB_EMPLOYEE

```

EMP_ID	EXPERTISE	ROW_NUM
1 5003	ENGLISH	1
2 107	ENGLISH	2
3 6003	ENGLISH	3
4 7003	ENGLISH	4
5 104	ENGLISH	5
6 105	ENGLISH	6
7 106	ENGLISH	7
8 5003	MATH	1
9 107	MATH	2
10 6003	MATH	3
11 104	MATH	4
12 7003	MATH	5
13 105	MATH	6
14 106	MATH	7
15 5003	SCIENCE	1

Handwritten annotations: A red bracket groups rows 1-7 (ENGLISH) and is labeled '1'. A blue bracket groups rows 8-14 (MATH) and is labeled '2'. An arrow points from row 15 (SCIENCE) to '3'.

Worksheet Query Builder

```

1 select A.*, row_number() over(partition by dept_id,a.expertise order by salary desc) rn from ab_employee A
2

```

Handwritten annotations: Circled numbers 1, 2, and 3 are placed under 'dept_id,a.expertise', 'order by salary desc', and 'rn' respectively. Arrows point from these circles to the corresponding parts of the SQL query.

Script Output x Query Result x

All Rows Fetched: 15 in 0.006 seconds

EMP_ID	EMP_NAME	DEPT_ID	EXPERTISE	SALARY	RESULTS	RN
1 5003	ABINASH	1	ENGLISH	50000	PASS	1
2 5003	ABINASH	1	MATH	50000	PASS	1
3 5003	ABINASH	1	SCIENCE	50000	PASS	1
4 106	REDDY	2	ENGLISH	80000	PASS	1
5 7003	NISHAD	2	ENGLISH	70000	FAIL	2
6 104	RAVI	2	ENGLISH	70000	PASS	3
7 6003	RAKESH	2	ENGLISH	50556	FAIL	4
8 107	AMARESH	2	ENGLISH	50000	PASS	5
9 106	REDDY	2	MATH	80000	FAIL	1
10 7003	NISHAD	2	MATH	70000	PASS	2
11 104	RAVI	2	MATH	70000	PASS	3
12 6003	RAKESH	2	MATH	50556	PASS	4
13 107	AMARESH	2	MATH	50000	PASS	5
14 105	JYOTI	3	ENGLISH	75000	PASS	1
15 105	JYOTI	3	MATH	75000	FAIL	1

Handwritten annotations: A blue arrow points from the 'RN' column to the text 'Create a boundary based on 2 Columns'.

See the above example, It is written `row_number() over(partition by dept_id,a.expertise order by salary desc)`

What does it means, 1st do partition by dept_id,a.expertise means, create a boundary based on these 2 column, after that arrange the data based on salary desc and in last row_number() will generate sequence 1,2,3,...

ROW_NUMBER Example 1

Create a row number by length

```

SELECT
  F.film_id,
  F.title,
  F.length,
  ROW_NUMBER() OVER(ORDER BY
length DESC) AS row_num
FROM film F
ORDER BY row_number;

```

Handwritten annotations: 'No Partition by no boundary' with an arrow pointing to the OVER clause. 'Arrange the data by length' with an arrow pointing to the ORDER BY clause in the OVER function.

film_id	title	length	row_num
182	CONTROL ANTHEM	185	1
141	CHICAGO NORTH	185	2
817	SOLDIERS EVOLUTION	185	3
212	DARN FORRESTER	185	4
349	GANGS PRIDE	185	5
...	...	...	...
469	IRON MOON	46	996
730	RIDGEMONT SUBMARINE	46	997
504	KWAI HOMEWARD	46	998
505	LABYRINTH LEAGUE	44	999
15	ALIEN CENTER	43	1000

Handwritten annotations: A bracket on the right side of the table is labeled 'Number generated 1, 2, 3, ...'.

As you can see, the ROW_NUMBER function generates a sequence of integers, starting from 1, for each row. But movies with the same lengths were given a DIFFERENT row number. Notice that there is no PARTITION BY clause because I am not grouping those movies into any sub-categories

ROW_NUMBER Example 2

Create a row number by length in a category

```
SELECT
  F.film_id,
  F.title,
  F.length,
  C.name AS category,
  ROW_NUMBER() OVER(PARTITION BY C.name
    ORDER BY F.length DESC) row_num
FROM film F
INNER JOIN film_category FC
ON FC.film_id = F.film_id
INNER JOIN category C
ON C.category_id = FC.category_id
ORDER BY C.name, row_number;
```

Don is here. it executes first & create a boundary

After that order the data within boundary

one C.name change start again

film_id	title	length	name	row_num
991	WORST BANGER	185	Action	1
212	DARN FORRESTER	185	Action	2
126	CASUALTIES ENCINO	179	Action	3
707	QUEST MUSSOLINI	177	Action	4
287	ENTRAPMENT SATISFACTION	176	Action	5
...				
690	POND SEATTLE	185	Animation	1
349	GANGS PRIDE	185	Animation	2
886	THEORY MERMAID	184	Animation	3
820	SONS INTERVIEW	184	Animation	4
510	LAWLESS VISION	181	Animation	5

Name Change

Number start again

Rank() and Dense_Rank()

RANK gives you the ranking within your ordered partition. Ties are assigned the same rank, with the next ranking(s) skipped. So, if you have 3 items at rank 2, the next rank listed would be ranked 5.

DENSE_RANK again gives you the ranking within your ordered partition, but the ranks are consecutive. No ranks are skipped if there are ranks with multiple items.

in case of rank, the number will be missed if the same number is repeated. but in case of dense_rank it is not missed.

like
if have salary 200
200
300

then rank will give as 1,1,3 --> skip the number sequence. 2 is missed here.
dense_rank will give you 1,1,2 --> not skip the number sequence

	Rank	DR
100	1	1
88	2	2
88	2	2
78	4	3
65	5	4
65	5	4
52	7	5
52	7	5
50	9	6

6 is skipped

8 is skipped

```
select emp_id, dept_id, EXPERTISE, SALARY,
row_number() over (partition by dept_id order by salary desc) row_num,
rank() over (partition by dept_id order by salary desc) ranking,
dense_rank() over (partition by dept_id order by salary desc) denseranking
from ab_employee;
```

9 `select emp_id, dept_id, EXPERTISE, SALARY,`
10 `row_number() over (partition by dept_id order by salary desc) row_num,`
11 `rank() over (partition by dept_id order by salary desc) ranking,`
12 `dense_rank() over (partition by dept_id order by salary desc) denseranking`
13 `from ab_employee;`

Query Result 3 x Query Result 4 x
All Rows Fetched: 15 in 0.024 seconds

EMP_ID DEPT_ID EXPERTISE SALARY ROW_NUM RANKING DENSERANKING

1	5003	1	SCIENCE	50000	1	1	1
2	5003	1	ENGLISH	50000	2	1	1
3	5003	1	MATH	50000	3	1	1
4	106	2	MATH	80000	1	1	1
5	106	2	ENGLISH	80000	2	1	1
6	104	2	ENGLISH	70000	3	3	2
7	104	2	MATH	70000	4	3	2
8	7003	2	ENGLISH	70000	5	3	2
9	7003	2	MATH	70000	6	3	2
10	6003	2	MATH	50556	7	7	3
11	6003	2	ENGLISH	50556	8	7	3
12	107	2	ENGLISH	50000	9	9	4
13	107	2	MATH	50000	10	9	4
14	105	3	ENGLISH	75000	1	1	1
15	105	3	MATH	75000	2	1	1

See the above Example and lets understand again 1 by 1

1. `row_number() over (partition by dept_id order by salary desc) row_num,`

- Row_Num is an alias, means nickname. Partition by dept_id, means once department change starts again. What start again? Doing Order and giving numbers 1,2,3,4.....
- In dept_id = 1 we have 3 rows, so it gives number 1,2,3 and then dept_id = 2 starts, which means department changes, so again, it starts again. take all records of department id 2, arrange them by order of salary in desc order and the start assign number. Total 10 rows for dept_id = 2, so 1,2,3,4,5,6,7,8,9,10, then it changes because dept_id = 3 starts.

2. `rank() over (partition by dept_id order by salary desc) ranking,`

- ranking is an alias, meaning nickname. Partition by dept_id, means once department change starts again. What start again? Doing Order and giving rankings 1,2,3,4.....
- In dept_id = 1 we have 3 rows, so first arrange them by order and they all have the same salary, so assign the same ranking 1,1,1 to all 3 records of dept_id = 1. and then dept_id = 2 starts, which means department changes, so again, it starts again. take all records of department id 2, arrange them by order of salary in desc order and then start assign ranking.
- First 2 records, same salary 80000, 80000 . So they both get the same ranking 1,1.
- Next record has a salary = 70000. so this got rank = 3. Here we skip the rank 2, because it is utilized by previous records. we just saw that we have 2 records of 80000 , so both get 1, so number 2 skipped and 70000 get the rank = 3
- Now the next 4 records have the same salary 70000, so all have the same rank 3 to each.
- Next record is 50556 salary and it comes after 4 times 70000. So 1st 70000 is rank = 3 and next 3 record wasted the rank 4,5,6. they all skipped and 50556 got rank = 7

3. `dense_rank() over (partition by dept_id order by salary desc) denseranking`

- denseranking is an alias, meaning nickname. Partition by dept_id, means once department change starts again. What start again? Doing Order and giving rankings 1,2,3,4.....
- In dept_id = 1 we have 3 rows, so first arrange them by order and they all have the same salary, so assign the same ranking 1,1,1 to all 3 records of dept_id = 1. and then dept_id = 2 starts, which means department changes, so again, it starts again. take all records of department id 2, arrange them by order of salary in desc order and then start assign ranking.
- First 2 records, same salary 80000, 80000 . So they both get the same ranking 1,1.
- Next record has a salary = 70000. so this got dense rank = 2. Here we **do not skip** the rank 2, **This is difference between rank and dense rank. Rank will skip the sequence and dense_rank will not.** So, here 70000 get dense_rank = 2
- Now next 4 records have the same salary 70000, so all have the same rank 2 to each.
- Next record is 50556 salary and it comes after 4 times 70000. So all 70000 have dense_rank = 2 and this assign dense_rank = 3

Worksheet Query Builder

```

1 select emp_id, dept_id, salary, expertise,
2 rank() over(partition by EXPERTISE order by salary desc) as rank1,
3 rank() over(partition by dept_id order by salary desc) as rank2
4 from ab_employee;
5

```

Query Result 3 x Query Result 4 x

All Rows Fetched: 15 in 0.005 seconds

EMP_ID	DEPT_ID	SALARY	EXPERTISE	RANK1	RANK2
1 106	2	80000	ENGLISH	1	1
2 105	3	75000	ENGLISH	2	1
3 7003	2	70000	ENGLISH	3	3
4 104	2	70000	ENGLISH	3	3
5 6002	2	50556	ENGLISH	5	7
6 5003	1	50000	ENGLISH	6	1
7 107	2	50000	ENGLISH	6	9
8 106	2	80000	MATH	1	1
9 105	3	75000	MATH	2	1
10 104	2	70000	MATH	3	3
11 7003	2	70000	MATH	3	3
12 6003	2	50556	MATH	5	7
13 107	2	50000	MATH	6	9
14 5003	1	50000	MATH	6	1
15 5003	1	50000	SCIENCE	1	1

Handwritten notes:

- ① creates boundary by expertise
- ② All English together then all math then all science
- ③ Same 70000 so same Rank = 3
- ④ Get Rank = 5 because 4 is skipped
- ⑤ Same sal 50000 so same Rank = 6
- Here when expertise change start again
- Here when dept change start again
- Partition by expertise, it change to math now start again

Worksheet Query Builder

```

1 select emp_id, dept_id, salary, expertise,
2 rank() over(partition by EXPERTISE order by salary desc) as rank1,
3 rank() over(partition by dept_id order by salary desc) as rank2
4 from ab_employee;
5

```

Query Result 3 x Query Result 4 x

All Rows Fetched: 15 in 0.005 seconds

EMP_ID	DEPT_ID	SALARY	EXPERTISE	RANK1	RANK2
1 106	2	80000	ENGLISH	1	1
2 105	3	75000	ENGLISH	2	1
3 7003	2	70000	ENGLISH	3	3
4 104	2	70000	ENGLISH	3	3
5 6003	2	50556	ENGLISH	5	7
6 5003	1	50000	ENGLISH	6	1
7 107	2	50000	ENGLISH	6	9
8 106	2	80000	MATH	1	1
9 105	3	75000	MATH	2	1
10 104	2	70000	MATH	3	3
11 7003	2	70000	MATH	3	3
12 6003	2	50556	MATH	5	7
13 107	2	50000	MATH	6	9
14 5003	1	50000	MATH	6	1
15 5003	1	50000	SCIENCE	1	1

Handwritten notes:

- All dept=2 is one group
- These are 1 group
- dept=3 is one group
- once dept change start again
- dept=1; some sal, 50000 all get Rank=1
- dept=2; start assign Rank after sorting by sal desc 80000=1 (2 records, so Rank=2 is skipped)
- 70000=3 (4 records & all get 3)

--> analytical function behaves in same way, and as written in partition by, it behaves like a group by and start again when that partition by column value changes.

for example: in case of RANK() function, it takes for dept_id = 1 and all 3 have the same salary, so it assigns value 1 to every one.

next record, dept_id is changed to 2, so it starts the ranking again. and for dept_id = 2, we have 2 records for salary = 80000, so it assign both as rank = 1, but next record is different salary but assigned rank = 3, because position of 2 is consumed. so next 4 records salary is same, so all have the same ranking 3,3,3,3 and when salary changes, it assign value = 7, because position of 4,5,6 is consumed by 3.

so here we can see that sequence is missed in case of RANK()

=====

once change start again

```
select E.*, rank() over(partition by DEPARTMENT_ID order by HIRE_DATE) joining_ranking from hr.employees E
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	JOINING_RANKI
200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-03	AD_ASST	4400	-	101	10	1
201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MAN	13000	-	100	20	1
202	Pat	Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000	-	201	20	2
114	Den	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	PU_MAN	11000	-	100	30	1
115	Alexander	Khoo	AKHOO	515.127.4562	18-MAY-03	PU_CLERK	3100	-	114	30	2
117	Sigal	Tobias	STOBIAS	515.127.4564	24-JUL-05	PU_CLERK	2800	-	114	30	3
116	Shelli	Baida	SBAIDA	515.127.4563	24-DEC-05	PU_CLERK	2900	-	114	30	4
118	Guy	Himuro	GHIHURO	515.127.4565	15-NOV-06	PU_CLERK	2600	-	114	30	5
119	Karen	Colmenares	KCOLMENEA	515.127.4566	10-AUG-07	PU_CLERK	2500	-	114	30	6
203	Susan	Mavris	SMAVRIS	515.123.7777	07-JUN-02	HR_REP	6500	-	101	40	1
122	Payam	Kaufling	PKAUFLIN	650.123.3234	01-MAY-03	ST_MAN	7900	-	100	50	1
137	Renske	Ladwig	RLADWIG	650.121.1234	14-JUL-03	ST_CLERK	3600	-	123	50	2

depends on Hire date we give Rank

```
with sneha as (
select E.*, rank() over(partition by department_id order by hire_date) joining_order from hr.employees E
) select * from sneha where joining_order = 2
```

all where =

LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	JOINING_ORDER
Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000	-	201	20	2
Khoo	AKHOO	515.127.4562	18-MAY-03	PU_CLERK	3100	-	114	30	2
Ladwig	RLADWIG	650.121.1234	14-JUL-03	ST_CLERK	3600	-	123	50	2
Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT_PROG	9000	-	102	60	2
Sully	PSULLY	011.44.1345.929268	04-MAR-04	SA_REP	9500	.35	146	80	2
King	SKING	515.123.4567	17-JUN-03	AD PRES	24000	-	-	90	2
Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI_MGR	12008	-	101	100	2

```
with sneha as (
select E.*, rank() over(partition by department_id order by hire_date) joining_order from hr.employees E
) select * from sneha where joining_order = 3
```

Who joined 3rd

ST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	JOINING_ORDER
bias	STOBIAS	515.127.4564	24-JUL-05	PU_CLERK	2800	-	114	30	3
js	TRAJS	650.121.8009	17-OCT-03	ST_CLERK	3500	-	124	50	3
taballa	VPATABAL	590.423.4560	05-FEB-06	IT_PROG	4800	-	103	60	3
el	EABEL	011.44.1644.429267	11-MAY-04	SA_REP	11000	.3	149	80	3
chhar	NKOCHHAR	515.123.4568	21-SEP-05	AD_VP	17000	-	100	90	3
en	JCHEN	515.124.4269	28-SEP-05	FI_ACCOUNT	8200	-	108	100	3

Write a sql to find the employee who are taking more than average salary of their department?

1. Lets first find the avg salary of each department.
I have 2 options. I can use Aggregate or Analytical both.

Option 1: Aggregate

Select Department_id, Avg(Salary) Avg_Sal from hr.employees Group by Department_id

Give any name
 with zameer as (
 SELECT h.*, avg(salary) over(partition by DEPARTMENT_ID) avh_dept_wise from hr.employees h
) select * from zameer
 ↳ creates boundary

	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	AVH_DEPT_WISE
I	515.123.4444	17-SEP-03	AD_ASST	4400	-	101	10	4400
TE	515.123.5555	17-FEB-04	HR_MAN	13000	-	100	20	9500
	603.123.6666	17-AUG-05	HR_REP	6000	-	201	20	9500
IL	515.127.4561	07-DEC-02	PU_MAN	11000	-	100	30	4150
	515.127.4562	18-MAY-03	PU_CLERK	3100	-	114	30	4150
	515.127.4563	24-DEC-05	PU_CLERK	2900	-	114	30	4150

Nth highest

CREATE TABLE Employee (name varchar(10), salary int);

INSERT INTO Employee VALUES ('Mr. X', 3000);
 INSERT INTO Employee VALUES ('Mr. Y', 4000);
 INSERT INTO Employee VALUES ('Mr. A', 3000);
 INSERT INTO Employee VALUES ('Mr. B', 5000);
 INSERT INTO Employee VALUES ('Mr. C', 7000);
 INSERT INTO Employee VALUES ('Mr. D', 1000);

select e.*, row_number() over (order by salary desc) as row_num from Employee e

select * from (select e.*, row_number() over (order by salary desc) as row_num from Employee e) where row_num = 4;

select * from (select e.*, rank() over (order by salary desc) as rank from Employee e) where rank = 3;

select * from (select e.*, dense_rank() over (order by salary desc) as rank from Employee e) where rank = 3;

select E.*, row_number() over(order by salary desc) row_num,
 rank() over(order by salary desc) ranking,
 dense_rank() over(order by salary desc) dense_ranking from emp E

NAME	SALARY	ROW_NUM	RANKING	DENSE_RANKING
B	7000	1	1	1
A	5000	2	2	2
Y	4000	3	3	3
X	3000	4	4	4
Z	3000	5	4	4
C	1000	6	6	5

6 rows selected.

SKIP

not skipped


```
with sana as (
select E.*,
dense_rank() over(order by salary desc) veda from emp E
) select * from sana where veda = 2
```

→ 2nd highest

NAME	SALARY	VEDA
A	5000	2

```
with sana as (
select E.*,
dense_rank() over(order by salary desc) veda from emp E
) select * from sana where veda = 3
```

3rd highest

NAME	SALARY	VEDA
Y	4000	3

```
insert into employee values ('Mr. E', null)
```

1 row(s) inserted.

↑ insert one null value

```
select * from employee
```

NAME	SALARY
Mr. E	-
Mr. X	3000
Mr. Y	4000
Mr. A	3000
Mr. B	5000
Mr. C	7000
Mr. D	1000

7 rows selected.

Now, there is nothing related to dense_rank() or rank(). This is all about how order by handle this. Remember that in Analytical function, order by will execute first and analytical after that. So, all thing is how order by handles null.

Bt default, Null will be first if DESC and last if ASC.(open my notes of order by, if any question on order by).

```
select E.*, dense_rank() over(order by salary desc) xx
from employee E
```

NAME	SALARY	XX
Mr. E	-	1
Mr. C	7000	2
Mr. B	5000	3
Mr. Y	4000	4
Mr. X	3000	5
Mr. A	3000	5
Mr. D	1000	6

7 rows selected.

```
select E.*, dense_rank() over(order by salary desc nulls last) xx
from employee E
```

NAME	SALARY	XX
Mr. C	7000	1
Mr. B	5000	2
Mr. Y	4000	3
Mr. X	3000	4
Mr. A	3000	4
Mr. D	1000	5
Mr. E	-	6

7 rows selected.

as it is



What happens if we insert 2 times null? How dense_rank() or rank() define that?

```

1 select E.*, dense_rank() over(order by salary desc) xx
2 from employee E

```

NAME	SALARY	XX
Mr. E	-	1
Mr. F	-	1
Mr. C	7000	2
Mr. B	5000	3
Mr. Y	4000	4
Mr. X	3000	5
Mr. A	3000	5
Mr. D	1000	6

Download CSV

8 rows selected.

> Both null consider same and assign rank 1

RANK Example 1

Create a ranking index by length

SELECT

F.film_id,

F.title,

F.length,

RANK() OVER(ORDER BY length
DESC) AS ranking

FROM film F

ORDER BY ranking;

All same length
behave same

film_id	title	length	ranking
182	CONTROL ANTHEM	185	1
141	CHICAGO NORTH	185	1
817	SOLDIERS EVOLUTION	185	1
212	DARN FORRESTER	185	1
349	GANGS PRIDE	185	1
609	MUSCLE BRIGHT	185	1
690	POND SEATTLE	185	1
872	SWEET BROTHERHOOD	185	1
991	WORST BANGER	185	1
426	HOME PITY	185	1
886	THEORY MERMAID	184	11
597	MOONWALKER FOOL	184	11
821	SORORITY QUEEN	184	11
813	SMOOCY CONTROL	184	11
820	SONS INTERVIEW	184	11
198	CRYSTAL BREAKING	184	11
180	CONSPIRACY SPIRIT	184	11
499	KING EVOLUTION	184	11
996	YOUNG LANGUAGE	183	19
469	IRON MOON	46	996
730	RIDGEMONT SUBMARINE	46	996
504	KWAH HOMEWARD	46	996
505	LABYRINTH LEAGUE	44	999
15	ALIEN CENTER	43	1000

data scientist school.com

All same length, so same Rank

2 to 10 is skipped so next value is 11

12-18 skipped bcoz same value so next is 19

RANK Example 2

Create a ranking index by length in a category

```
SELECT
  F.film_id,
  F.title,
  F.length,
  C.name AS category,
  RANK() OVER(PARTITION BY C.name ORDER BY
    F.length DESC) ranking
FROM film F
INNER JOIN film_category FC
ON FC.film_id = F.film_id
INNER JOIN category C
ON C.category_id = FC.category_id
ORDER BY C.name, ranking;
```

film_id	title	length	category	ranking
991	WORST BANGER	185	Action	1
212	DARN FORRESTER	185	Action	1
126	CASUALTIES ENCINO	179	Action	3
707	QUEST MUSSOLINI	177	Action	4
287	ENTRAPMENT	176	Action	5
...				
690	POND SEATTLE	185	Animation	1
349	GANGS PRIDE	185	Animation	1
886	THEORY MERMAID	184	Animation	3
820	SONS INTERVIEW	184	Animation	3
510	LAWLESS VISION	181	Animation	5

→ same length so both are 1
→ 2 is skipped
→ category change, start again

DENSE_RANK Example 1

Create a dense ranking index by length

```
SELECT
  F.film_id,
  F.title,
  F.length,
  DENSE_RANK() OVER(ORDER BY
    length DESC) AS ranking
FROM film F
ORDER BY ranking;
```

film_id	title	length	ranking
182	CONTROL ANTHEM	185	1
141	CHICAGO NORTH	185	1
817	SOLDIERS EVOLUTION	185	1
212	DARN FORRESTER	185	1
349	GANGS PRIDE	185	1
609	MUSCLE BRIGHT	185	1
690	POND SEATTLE	185	1
872	SWEET BROTHERHOOD	185	1
991	WORST BANGER	185	1
426	HOME PITY	185	1
886	THEORY MERMAID	184	2
597	MOONWALKER FOOL	184	2
821	SORORITY QUEEN	184	2
...			
398	HANOVER GALAXY	47	139
869	SUSPECTS QUILLS	47	139
469	IRON MOON	46	140
730	RIDGEMONT SUBMARINE	46	140
504	KWAI HOMEWARD	46	140
505	LABYRINTH LEAGUE	44	141
15	ALIEN CENTER	43	142

→ All same so 1

→ next value is 2

DENSE_RANK Example 2

Create a dense rank index by length in a category

```
SELECT
  F.film_id,
  F.title,
  F.length,
  C.name AS category,
  DENSE_RANK() OVER(PARTITION BY C.name
    ORDER BY F.length DESC) ranking
FROM film F
INNER JOIN film_category FC
ON FC.film_id = F.film_id
INNER JOIN category C
ON C.category_id = FC.category_id
ORDER BY C.name, ranking;
```

once change, start again

film_id	title	length	category	ranking
991	WORST BANGER	185	Action	1
212	DARN FORRESTER	185	Action	1
126	CASUALTIES ENCINO	179	Action	2
707	QUEST MUSSOLINI	177	Action	3
287	ENTRAPMENT	176	Action	4
511	LAWRENCE LOVE	175	Action	5
927	UPRISING UPTOWN	174	Action	6
549	MAGNOLIA	171	Action	7
250	DRAGON SQUAD	170	Action	8
...				
811	SMILE EARRING	60	Travel	43
981	WOLVES DESIRE	55	Travel	44
386	GUMP DATE	53	Travel	45
931	VALENTINE	48	Travel	46
784	SHANGHAI TYCOON	47	Travel	47

same length so 1

not skipped

category change, start again